

OPERATING SYSTEMS

Chapter 8: DEADLOCKS

Complete University Exam Notes

Topics Covered

System Model • Deadlock Characterization • Four Necessary Conditions
Resource Allocation Graphs • Handling Methods • Prevention Strategies
Ostrich Algorithm • Resource Ordering • Recovery Techniques

Based on: Operating System Concepts – 9th Edition (Silberschatz, Galvin & Gagne)

1. System Model

1.1 Definition

A system model describes how processes use resources in an operating system. Before a process can use a resource, it must request it. After using it, it must release it.

1.2 Key Components

- The system has many types of resources.
- Resources include: CPU cycles, memory space, I/O devices (like printers, disks).
- Each resource type R_i has W_i instances (copies).
- Example: If there are 3 printers, the printer resource type has 3 instances.

1.3 How a Process Uses a Resource (3 Steps)

Step	Action	Description
Step 1	Request	Process asks the OS for the resource. If not available, process waits.
Step 2	Use	Process uses the resource to do its work.

Step 3	Release	Process finishes and gives back the resource to OS.
--------	---------	---

★ Important Exam Points

- Resource types: $R_1, R_2, \dots, R_m$
- Each type has W_i instances
- Three steps: Request $\rightarrow$ Use $\rightarrow$ Release

2. Deadlock

2.1 Definition

A deadlock is a situation where two or more processes are waiting for each other to release resources — but none of them ever will. All processes are stuck permanently.

2.2 Simple Real-Life Analogy

- Two cars on a narrow one-lane road facing each other.
- Neither can move forward. Neither will reverse. Both are stuck.
- This is exactly like a deadlock in OS.

2.3 Classic OS Example

```
Program 1 requests Resource A and receives it.
Program 2 requests Resource B and receives it.
Program 1 requests Resource B — WAITS (B held by Program 2).
Program 2 requests Resource A — WAITS (A held by Program 1).
```

Result: Both programs wait forever. DEADLOCK!

3. Deadlock Characterization — Four Necessary Conditions

Deadlock can ONLY happen when ALL FOUR conditions hold at the same time. If even ONE condition is missing $\rightarrow$ NO deadlock.

3.1 The Four Coffman Conditions

#	Condition	Simple Meaning
1	Mutual Exclusion	Only ONE process can use a resource at a time. Others must wait.
2	Hold and Wait	A process holds one resource AND is waiting to get more.
3	No Preemption	Resources cannot be forcibly taken away. A process releases voluntarily.
4	Circular Wait	Process A waits for B, B waits for C, C waits for A — a circle!

3.2 Mutual Exclusion — Detailed

- Only one process can use a resource at any given time.
- Non-sharable resources: like a printer — only one process prints at a time.
- Sharable resources: like read-only files — many processes can read at the same time.
- Mutual exclusion is NEEDED for non-sharable resources.

3.3 Hold and Wait — Detailed

- Process holds at least 1 resource.
- At the same time, it is waiting to get additional resources held by other processes.
- Example: A process holds a printer but is waiting for a scanner.

3.4 No Preemption — Detailed

- OS cannot forcibly take a resource from a process.
- The process must voluntarily release it after completing its task.
- Example: A process printing cannot have the printer snatched away mid-print.

3.5 Circular Wait — Detailed

- A set of processes $\{P_0, P_1, P_2, \dots, P_n\}$ where:
 - P_0 waits for a resource held by P_1
 - P_1 waits for a resource held by P_2
 - ... and so on ...
 - P_n waits for a resource held by $P_0 \rightarrow$ CIRCLE!

★ **KEY RULE: ALL 4 conditions must exist together for deadlock. Prevent even ONE $\rightarrow$ No deadlock!**

4. Resource Allocation Graph (RAG)

4.1 Definition

A Resource Allocation Graph (RAG) is a directed graph used to describe the state of resource allocation in a system. It shows which process holds which resource and which process is waiting for which resource.

4.2 Components of RAG

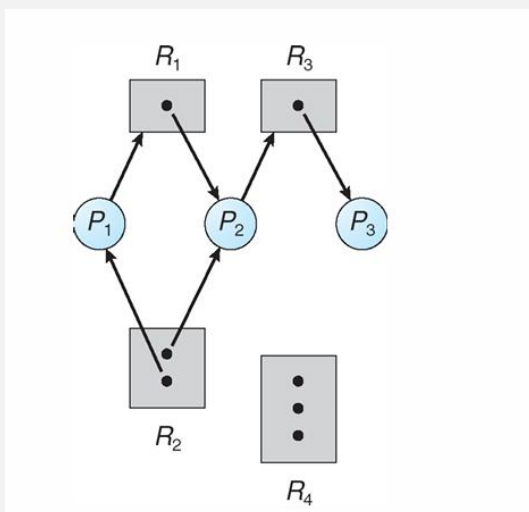
Component	Symbol	Description
Process	Circle (P_i)	Represents a process in the system
Resource Type	Rectangle (R_j)	Represents a type of resource; dots inside = instances
Request Edge	$P_i \rightarrow R_j$	Process P_i is requesting (waiting for) resource R_j
Assignment Edge	$R_j \rightarrow P_i$	Resource R_j is assigned to (held by) process P_i

4.3 Request Edge vs Assignment Edge

Type	Direction	Meaning	Example
Request Edge	Process $\rightarrow$ Resource	Process is WAITING for the resource	$P_1 \rightarrow R_1$ means P_1 wants R_1
Assignment Edge	Resource $\rightarrow$ Process	Resource is GIVEN to the process	$R_1 \rightarrow P_1$ means R_1 is held by P_1

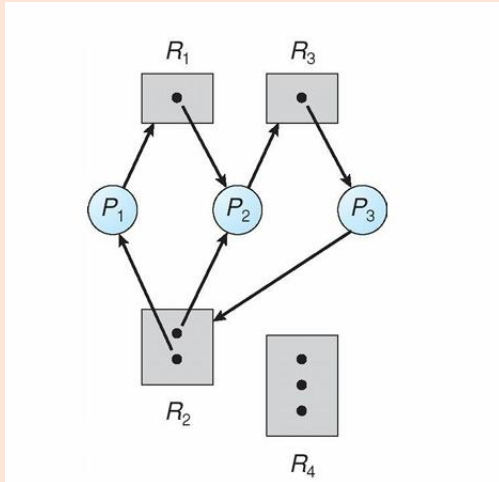
4.4 Diagram Reference

Example of a Resource Allocation Graph]



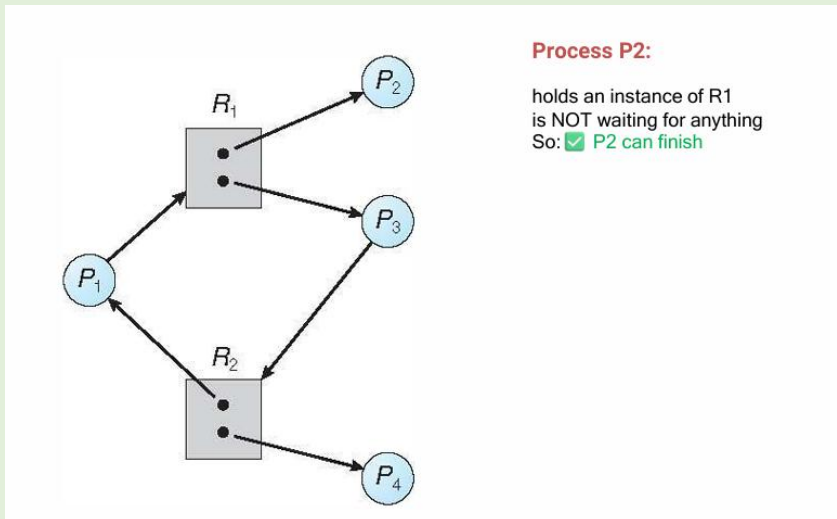
The diagram shows processes P_1 , P_2 , P_3 and resources R_1 , R_2 , R_3 , R_4 with request and assignment edges.

Resource Allocation Graph With A Deadlock]



This diagram shows a cycle forming between processes and resources, causing a deadlock situation.

Graph With A Cycle But No Deadlock]



This shows a cycle exists but Process P2 (which holds R1) is NOT waiting → P2 can finish → releases R1 → cycle breaks → NO deadlock.

5. Basic Facts About Deadlock and RAG

Situation	Conclusion
Graph has NO cycle	→ DEFINITELY NO DEADLOCK

Graph has a cycle + only 1 instance per resource	→ DEFINITELY DEADLOCK
Graph has a cycle + multiple instances per resource	→ POSSIBLE DEADLOCK (not guaranteed)

5.1 Cycle vs Deadlock — Common Confusion Explained

- A cycle in RAG does NOT always mean deadlock.
- If resources have multiple instances, a process outside the cycle might finish and release a resource.
- That released resource can break the cycle → No deadlock.
- Single instance resources: Cycle = Deadlock (always).
- Multiple instance resources: Cycle = Possible Deadlock (check carefully).

★ **Exam Tip: Cycle is NECESSARY but not SUFFICIENT for deadlock (when multiple instances exist).**

6. Methods for Handling Deadlocks

6.1 Three Main Approaches

Approach	Strategy	Used In
Prevention / Avoidance	Ensure system NEVER enters deadlock state	High safety systems
Detection + Recovery	Allow deadlock to occur, then detect and recover	General purpose OS
Ignore (Ostrich Algorithm)	Pretend deadlocks don't exist	UNIX, Windows (rare deadlocks)

6.2 Ostrich Algorithm

The Ostrich Algorithm = IGNORE the deadlock problem completely. Just like an ostrich burying its head in the sand and pretending there is no danger.

- Assumption: Deadlocks are so rare that preventing them is not worth the cost.
- If deadlock happens → OS reboots to recover.
- Used by: UNIX, Windows (in practice).
- Reason: Deadlocks happen very rarely in real systems.

- Trade-off: Saves resources in normal operation; can cause problems when deadlock occurs.

★ Exam Points

- Ostrich = Ignore + Reboot approach
- Used when deadlocks are rare and prevention cost is too high
- NOT the safest option but MOST practical for general-purpose OS

7. Deadlock Prevention

Deadlock Prevention = Make sure at least ONE of the four necessary conditions CANNOT happen. If one condition is blocked → Deadlock is impossible!

7.1 Overview of Prevention Methods

Condition to Attack	Method Used	Result
Mutual Exclusion	Make resources sharable (spooling)	Fewer exclusive locks
Hold and Wait	Request all resources upfront OR release before new request	No holding while waiting
No Preemption	Allow resource preemption / virtualization	OS can take resources back
Circular Wait	Assign resource ordering / numbering	No cycles possible

7.2 Attacking Mutual Exclusion

Idea

- If resources can be shared, processes don't need exclusive access.
- Sharing reduces waiting and eliminates the mutual exclusion condition.

Printer Spooling — Key Example

Concept	Explanation
Problem	If two processes directly access a printer → conflict
Solution	Spooling: Processes write to a disk spool file, not directly to printer
Printer Daemon	A background program reads the spool file and prints one job at a time
Advantage	Multiple processes can 'use' printer simultaneously (via spool)

Limitation	Some resources CANNOT be made sharable (e.g., two processes writing to same file simultaneously causes corruption)
-------------------	--

Read-Only Files Example

- Multiple processes can read the same file at the same time.
- No conflict because no one is writing.
- Mutual exclusion not needed for read-only access.

★ **Limitation: Cannot eliminate mutual exclusion for all resources. Some resources like printers, locks, database records must remain exclusive.**

7.3 Attacking Hold and Wait

Method 1: Request All Resources Before Execution Starts

1. Process must request ALL resources it will ever need BEFORE it starts running.
2. If ALL resources are available → Allocate all and let process run.
3. If even ONE resource is not available → Process waits (holding nothing).

Advantages	Disadvantages
Eliminates hold-and-wait completely	Poor resource utilization
Simple to implement	Processes may not know all future resource needs
Prevents deadlock	Resources are held unnecessarily for a long time
	Starvation possible (process may wait forever if popular resources always busy)

Method 2 (Alternative): Release Before Requesting

- If a process needs a new resource, it must FIRST release all resources it currently holds.
- Then it requests everything it needs again.

Step	Action
1	Process releases ALL currently held resources
2	Process requests ALL resources it needs (old + new)
3	If all available → allocate and run. If not → keep waiting.

- Advantage: Eliminates circular dependency.
- Disadvantage: Frequent releasing/reacquiring causes overhead and inefficiency.

7.4 Attacking No Preemption

Normally, resources cannot be forcibly taken from a process. Attacking this condition means allowing the OS to take (preempt) resources when needed.

Approach: Virtualization

- Instead of directly using hardware, create a virtual version.
- Example: Printer Spooling — processes write to virtual printer (disk spool), not real printer.
- Printer daemon manages the real printer without conflict.

Aspect	Details
Advantage	Reduces direct contention for physical hardware
Limitation	Not all resources can be virtualized
Cannot Virtualize	Database records, OS tables — they must be locked for use, and locking itself can cause deadlock
Safe to Virtualize	Printers, plotters, display devices (via spooling)

7.5 Attacking Circular Wait

This is the MOST PRACTICAL prevention method. The idea is to prevent any circular chain of waiting from forming.

Method 1: One Resource at a Time

- A process may hold only ONE resource at a time.
- If it needs another resource, it must first release the current one.
- Limitation: Impractical for many real tasks.
- Example: Copying from tape drive to printer — needs BOTH simultaneously. Cannot work with one at a time.

Method 2: Resource Ordering (Most Important!)

Assign a unique number to every resource type. Processes must request resources ONLY in increasing order of their numbers.

7.6 Resource Ordering Technique — Detailed

How It Works

Resource	Assigned Number
Imagewriter	1
Printer	2
Plotter	3
Tape Drive	4
Blu-ray Drive	5

Rule

A process can only request resource R_j if R_j 's number is GREATER than the number of ANY resource currently held.

Examples

Request	Allowed?	Reason
Printer (2) → Tape Drive (4)	✓ ALLOWED	$4 > 2$, increasing order
Plotter (3) → Printer (2)	✗ NOT ALLOWED	$2 < 3$, decreasing order, must release Plotter first
Tape Drive (4) → Blu-ray (5)	✓ ALLOWED	$5 > 4$, increasing order

Why Does This Prevent Circular Wait?

Proof by contradiction:

- Suppose Process A holds resource i and waits for resource j ($j > i$ — required by rule)
- For a cycle, Process B must hold j and wait for i
- But $i < j$ → Process B cannot request i (it would be requesting LOWER numbered resource)
- **CONTRADICTION!** → No cycle can form → No circular wait → No deadlock

Aspect	Details
Advantage	Effective — completely eliminates circular wait. Relatively easy to implement.
Disadvantage	Resource numbering must be carefully planned. Processes must know resource order in advance.
Key Point	No cycle = No deadlock (guaranteed)

8. ONE PAGE QUICK REVISION SHEET

Topic	Key Points
Deadlock	Two+ processes waiting for each other's resources. No one moves. All stuck permanently.
4 Conditions	Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait (ALL must hold)
RAG Request Edge RAG Assignment Edge Cycle + 1 instance	Process → Resource (process is WAITING) Resource → Process (resource is HELD) = Definite DEADLOCK
Cycle + multiple instances	= Possible deadlock (check if any process can finish)
Ostrich Algorithm	Ignore deadlocks; reboot if it happens; used in UNIX/Windows
Prevention: Mutual Excl.	Use spooling (printer daemon); make resources sharable
Prevention: Hold & Wait	Request all resources before starting OR release all before requesting more
Prevention: No Preemption	Virtualize resources (spooling); OS can reclaim virtualized resources
Prevention: Circular Wait	Assign numbers to resources; always request in increasing order

9. Important Definitions Table

Term	Definition
Deadlock	A permanent standstill where processes wait for resources held by each other.
Resource Instance	A single copy of a resource type (e.g., one of three printers).
Mutual Exclusion	Only one process can use a non-sharable resource at a time.
Hold and Wait	A process holds resources while waiting to acquire more.
No Preemption	Resources cannot be forcibly taken; only voluntarily released.
Circular Wait	A circular chain of processes each waiting for a resource held by the next.
RAG	Resource Allocation Graph — directed graph showing resource allocation state.
Request Edge	Directed edge from Process to Resource — process is waiting for resource.

Assignment Edge	Directed edge from Resource to Process — process holds the resource.
Ostrich Algorithm	Strategy of ignoring deadlocks; assume they are rare enough to ignore.
Spooling	Technique where process output is stored on disk; daemon handles actual device.
Resource Ordering	Assigning numbers to resources and requiring requests in increasing order.
Starvation	A process waits indefinitely because resources are always taken by others.
Preemption	Forcibly removing a resource from a process (used in prevention).
Safe State	System state where a safe sequence exists and deadlock can be avoided.

10. Important Comparison Tables

10.1 Three Methods to Handle Deadlock

Method	Description	Advantage	Disadvantage
Prevention	Eliminate one of 4 conditions	Guaranteed no deadlock	Low resource utilization
Detection + Recovery	Allow deadlock, then fix it	Better resource use	Recovery cost, may lose work
Ignorance (Ostrich)	Ignore deadlocks completely	No overhead	System may crash or freeze

10.2 Prevention Methods Comparison

Condition Attacked	Method	Advantage	Disadvantage
Mutual Exclusion	Spooling, sharing	Reduces conflicts	Cannot work for all resources
Hold and Wait (M1)	Request all resources initially	Simple, no hold+wait	Poor utilization, starvation
Hold and Wait (M2)	Release before new request	Eliminates circular deps	Overhead, inefficiency
No Preemption	Virtualization/spooling	Reduces contention	Cannot virtualize all resources

Circular Wait	Resource ordering by number	Very effective	Needs careful planning
----------------------	-----------------------------	----------------	------------------------

10.3 Cycle in RAG — Deadlock or Not?

Resource Instances	Cycle Present?	Deadlock?
Single instance per type	YES	DEFINITELY YES
Single instance per type	NO	DEFINITELY NO
Multiple instances per type	YES	POSSIBLY (check if any process can finish)
Multiple instances per type	NO	DEFINITELY NO

11. Four Conditions Summary Table

Condition	Meaning	Prevention Strategy	Example
Mutual Exclusion	Only 1 process uses resource at a time	Spooling; sharing read-only files	Printer: only 1 process prints
Hold and Wait	Process holds resource + waits for more	Request all at start OR release all first	P holds printer, wants scanner
No Preemption	Resources cannot be forcibly removed	Virtualize resources; preempt if needed	Printing: cannot interrupt mid-page
Circular Wait	Processes wait in a circle for resources	Number resources; request in order only	P1→R1→P2→R2→P1 (cycle)

12. Possible Viva Questions

Question	Short Answer
What is a deadlock?	A state where processes wait for each other's resources indefinitely.
What are the 4 conditions for deadlock?	Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.

What is the Ostrich Algorithm?	Ignore deadlocks; assume they are rare. Used in UNIX/Windows.
What is a RAG?	Resource Allocation Graph — shows which process holds/waits for which resource.
What is a request edge?	An edge from process to resource — process is waiting for that resource.
What is an assignment edge?	An edge from resource to process — process holds that resource.
Does a cycle in RAG always mean deadlock?	No. Only if each resource has 1 instance. Multiple instances = possible but not certain.
How does resource ordering prevent deadlock?	By requiring requests in increasing order, no circular chain can form.
What is spooling? How does it help?	Jobs are written to disk; daemon prints them. Multiple processes share printer without conflict.
What is Hold & Wait prevention Method 2?	Release ALL resources before requesting any new ones.

13. Possible Exam Questions

13.1 Short Questions (2–5 marks)

4. Define deadlock. Give one example.
5. State the four necessary conditions for deadlock.
6. What is the Ostrich Algorithm? When is it used?
7. Differentiate between request edge and assignment edge in RAG.
8. What are the basic facts about cycles in a RAG?
9. Explain printer spooling and its role in deadlock prevention.
10. What is resource ordering? Give an example.
11. State any two methods to handle deadlocks.

13.2 Broad / Long Questions (10–15 marks)

12. Explain the four necessary conditions for deadlock with examples.
13. Explain Resource Allocation Graph with diagrams. When does a cycle indicate a deadlock?
14. Explain all methods of Deadlock Prevention in detail.
15. Compare and contrast the three approaches to handling deadlocks.
16. Explain how attacking the Circular Wait condition prevents deadlock. Give a proof.
17. What is the Hold and Wait condition? Give two methods to prevent it. Compare their advantages and disadvantages.

13.3 Common Tricky Exam Questions

Tricky Question	Correct Answer
Can a deadlock occur with only 2 processes?	YES — if they each hold one resource and request the other.
Does every cycle in RAG mean deadlock?	NO — only if each resource has exactly 1 instance.
Is Ostrich Algorithm the best strategy?	No — it's practical for rare deadlocks but not safe in critical systems.
If we prevent Circular Wait, is deadlock impossible?	YES — because all 4 conditions must hold; removing one prevents deadlock.
Can spooling prevent deadlock for all resources?	No — only for resources that can be virtualized (like printers).
Is starvation the same as deadlock?	No — in starvation, a process waits a long time but may eventually run. In deadlock, it waits forever.

14. Final Summary — Everything in One Place

DEADLOCK AT A GLANCE

Deadlock = Permanent standstill of processes waiting for resources.

4 Conditions (ALL must hold): Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait

3 Handling Methods: Prevention | Detection & Recovery | Ignore (Ostrich)

RAG: No cycle = No deadlock. Cycle + 1 instance = Deadlock. Cycle + multiple = Possible.

Best Prevention = Resource Ordering (assign numbers, request in increasing order only).

Spooling = processes write to disk spool, daemon handles actual device. Reduces conflict.

Good Luck on Your Exam! 🎓

Operating System Concepts – Chapter 8: Deadlocks